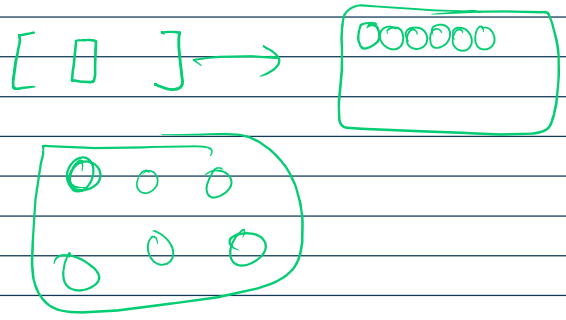
22 October 2026 19:20

```python
class Node:
  def __init__(self, data):
    self.data = data
    self.next = None
    self.prev = None
class doubly_linked_list:
  def __init__(self):
    self.head = None
    self.last = None
    #Write solution code here
    def insert_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            self.last = new_node
        else:
            self.last.next = new_node
            new_node.prev = self.last
            self.last = new_node
```



Empty Doubly linked list
self.head = new_node
self.last = new_node

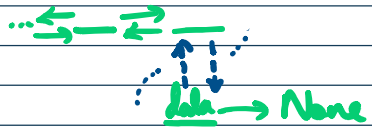
Not Empty

current l
new l



self.last.next = new_node
new_node.prev = self.last
self.last = new_node





Graph: ⇒ Nodes, Edges
= Vertices

⇒ Directed Edges, Undirected Edges

⇒ Acyclic graphs, Cyclic graph

⇒ Weights (on edges)

⇒ In-degree, Out-degree (Nodes / Vertices)

⇒ Path (collections of ordered edges/ vertices)
= Walk

⇒ Connectedness (Reachability)

⇒ Components

In code, two ways to represent graph $A_{ij} = a_{ji}$

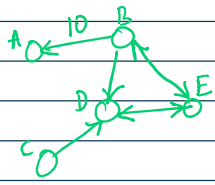
Adjacency Matrix



A B C D E

A [[0,0,0,0,0], [[0,1,0,0,0],

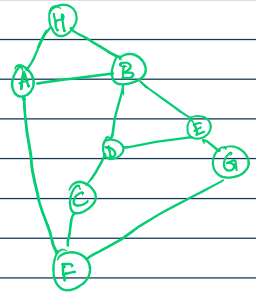
B [1,0,0,1,1], [1,0,0,1,1],



A	[[0,0,0,0,0],	[[0,1,0,0,0],
B	[1,0,0,1,1],	[1,0,0,1,1],
C	[0,0,0,1,0],	[0,0,0,1,0],
D	[0,0,0,0,1],	[0,1,1,0,1],
E	[0,1,0,1,0]]	[0,1,0,1,0]]

Adjacency List { A: [], B: [A,D,E], C: [D], D: [E], E: [B,D] }

Undirected: { A: [B], B: [A,D,E], C: [D], D: [B,C,E], E: [B,D] }



Node	Neighbours	Visited
B	A,D,E,H	B
A	D,E,H,F	

Node	Neighbours	Visited
B	A,D,E,H	B
A		

```python
def DFS(adjl, s, visited):
  if visited is None:
    visited = set()
  visited.add(s)
  print(f"{s}")
  for neigh in adjl[s]:
    if neigh not in visited:
      DFS(adjl, neigh, visited)
  return visited
```

```python
def DFS_Recursive(adjm, i, visited):
  if visited is None:
    visited = set()
  visited.add(i)
  print(i)
  for j in range(len(adjm)):
    if adjm[i][j] == 1 and j not in visited:
      DFS_Recursive(adjm, j, visited)
  return visited
```